

## MODÜL 02

# Aktivasyon Fonksiyonları

Activation Functions — Derinlemesine İnceleme

Sigmoid · Tanh · ReLU · Leaky ReLU · ELU · SELU · Swish · GELU

|               |                                                                |
|---------------|----------------------------------------------------------------|
| Seviye        | Başlangıç → İleri                                              |
| Kapsam        | 8 Fonksiyon · Vanishing Gradient · Dead Neuron · Karar Rehberi |
| Framework     | NumPy · TensorFlow/Keras · PyTorch                             |
| Önceki Modül  | Modül 01 — Sinir Ağları Temelleri                              |
| Sonraki Modül | Modül 03 — Kayıp Fonksiyonları ve Optimizasyon                 |

## Modül Konsept Haritası

|                   |   |                |   |                 |
|-------------------|---|----------------|---|-----------------|
| Aktivasyon Neden? | → | 8 Fonksiyon    | → | Seçim Rehberi   |
| Non-linearity     |   | formül+türev   |   | Durum bazlı     |
|                   |   | ↓              |   |                 |
|                   |   | Vanishing Grad | → | Dead Neuron     |
|                   |   | Sigmoid sorunu |   | ReLU sorunu     |
|                   |   | ↓              |   | ↓               |
| Modern Çözümler   | ← | GELU · Swish   |   | Leaky/ELU       |
| BERT · GPT std.   |   | Transformer    |   | ReLU alternatif |

## 1. Öğrenme Hedefleri

- ✓ 8 aktivasyon fonksiyonunun formülünü, türevini ve çıkış aralığını söylemek
- ✓ Vanishing gradient problemini matematiksel olarak kanıtlamak ve grafik üzerinde göstermek
- ✓ Dead Neuron (ölü nöron) problemini açıklamak ve çözümlerini uygulamak
- ✓ Sigmoid ve Tanh'ın neden derin ağlarda sorun çıkardığını türevle kanıtlamak
- ✓ GELU ve Swish'in neden modern mimarilerde tercih edildiğini açıklamak
- ✓ Hangi durumda hangi aktivasyonu seçmek gerektiğine dair karar rehberi oluşturmak
- ✓ Tüm aktivasyonları NumPy, TensorFlow ve PyTorch ile çalıştırılabilir şekilde kodlamak

## 2. Aktivasyon Fonksiyonları Neden Gerekli?

Aktivasyonsuz bir sinir ağını düşünelim. Her katman  $z = W \cdot x + b$  dönüşümü yapar. Bu dönüşümler birleşince yine lineer bir dönüşüm elde edilir — kaç katman olursa olsun. Matematiksel kanıt:  $W_2 \cdot (W_1 \cdot x + b_1) + b_2 = (W_2 W_1) \cdot x + (W_2 b_1 + b_2)$ . Tek bir matris çarpımıyla özetlenebilir. Yani derin ağın hiçbir avantajı kalmaz. Aktivasyon fonksiyonu bu döngüyü kırar — ağa doğrusal olmayan kapasite kazandırır.

## 3. Aktivasyon Fonksiyonları — Formül ve Analiz

### Sigmoid $\sigma(z)$

| Formül     | $\sigma(z) = 1 / (1 + e^{(-z)})$                                                                                               |
|------------|--------------------------------------------------------------------------------------------------------------------------------|
| Türev      | $\sigma'(z) = \sigma(z) \cdot (1 - \sigma(z))$ , maks. = 0.25 ( $z=0$ 'da)                                                     |
| Aralık     | (0, 1)                                                                                                                         |
| Avantaj    | İkili sınıflandırmada çıkış katmanı için idealdir. Çıktı olasılık yorumu taşır.                                                |
| Dezavantaj | Vanishing gradient: $z$ büyük/küçükken türev $\rightarrow 0$ . Sıfır merkezli değil $\rightarrow$ ağırlık güncellemeleri hep a |

### Tanh $\tanh(z)$

| Formül     | $\tanh(z) = (e^z - e^{(-z)}) / (e^z + e^{(-z)})$                                   |
|------------|------------------------------------------------------------------------------------|
| Türev      | $\tanh'(z) = 1 - \tanh^2(z)$ , maks. = 1.0 ( $z=0$ 'da)                            |
| Aralık     | (-1, 1)                                                                            |
| Avantaj    | Sıfır merkezlidir — Sigmoid'den daha iyi. RNN/LSTM gizli durumlarında standarttır. |
| Dezavantaj | Hâlâ vanishing gradient sorunu var (sigmoid'den az). Doyma noktaları mevcut.       |

### ReLU $\max(0, z)$

| Formül     | $\text{ReLU}(z) = \max(0, z)$                                                                                              |
|------------|----------------------------------------------------------------------------------------------------------------------------|
| Türev      | $\text{ReLU}'(z) = 1$ eğer $z > 0$ , $0$ eğer $z \leq 0$                                                                   |
| Aralık     | $[0, +\infty)$                                                                                                             |
| Avantaj    | Hızlı, basit. $z > 0$ 'da vanishing gradient YOK. Derin CNN'lerin standart tercihi.                                        |
| Dezavantaj | Dead Neuron: $z \leq 0$ olan nöron türev=0 alır $\rightarrow$ hiç güncellenmez $\rightarrow$ 'ölür'. Sıfır merkezli değil. |

## Leaky ReLU

| Formül     | $\text{Leaky ReLU}(z) = \max(\alpha z, z)$ [ $\alpha = 0.01$ varsayılan]          |
|------------|-----------------------------------------------------------------------------------|
| Türev      | $1$ eğer $z > 0$ , $\alpha$ eğer $z \leq 0$ ( $\alpha$ genellikle 0.01)           |
| Aralık     | $(-\infty, +\infty)$                                                              |
| Avantaj    | Dead Neuron problemini çözer: $z < 0$ 'da hâlâ küçük gradyan ( $\alpha$ ) vardır. |
| Dezavantaj | $\alpha$ hiperparametresi seçimi gerektirir. PReLU $\alpha$ 'yı öğrenir.          |

## ELU Exponential Linear Unit

| Formül     | $\text{ELU}(z) = z$ eğer $z > 0$ , $\alpha \cdot (e^z - 1)$ eğer $z \leq 0$ [ $\alpha = 1$ varsayılan] |
|------------|--------------------------------------------------------------------------------------------------------|
| Türev      | $1$ eğer $z > 0$ , $\text{ELU}(z) + \alpha$ eğer $z \leq 0$                                            |
| Aralık     | $(-\alpha, +\infty)$                                                                                   |
| Avantaj    | Smooth negatif doyma, sıfır merkezli, dead neuron yok. ReLU'dan daha iyi yakınsama.                    |
| Dezavantaj | Üstel hesaplama ReLU'dan yavaş.                                                                        |

## SELU Scaled ELU

| Formül     | $\text{SELU}(z) = \lambda \cdot \text{ELU}(z, \alpha)$ [ $\lambda \approx 1.0507$ , $\alpha \approx 1.6733$ — sabit!]       |
|------------|-----------------------------------------------------------------------------------------------------------------------------|
| Türev      | $\lambda \cdot (1$ veya $\alpha \cdot e^z)$                                                                                 |
| Aralık     | $(\approx -1.758, +\infty)$                                                                                                 |
| Avantaj    | Self-normalizing: Çıktılar otomatik olarak ortalaması $\rightarrow 0$ , varyansı $\rightarrow 1$ olur. Batch Norm gerekmez. |
| Dezavantaj | Yalnızca tam bağlantılı (fully-connected) katmanlarda tam çalışır. CNN'ler için önerilmez.                                  |

## Swish $z \cdot \sigma(z)$

| Formül     | $\text{Swish}(z) = z \cdot \sigma(z)$ [ $\beta = 1$ ]                                                 |
|------------|-------------------------------------------------------------------------------------------------------|
| Türev      | $\sigma(\beta z) + \beta z \cdot \sigma(\beta z) \cdot (1 - \sigma(\beta z))$                         |
| Aralık     | $(\approx -0.278, +\infty)$                                                                           |
| Avantaj    | Pürüzsüz, monoton değil. Büyük ölçekli görüntü sınıflandırmada ReLU'yu geçti. EfficientNet standardı. |
| Dezavantaj | Hesaplaması ReLU'dan biraz daha pahalı.                                                               |

## GELU Gaussian Error LU

| Formül     | $\text{GELU}(z) \approx 0.5 \cdot z \cdot [1 + \tanh(\sqrt{2/\pi} \cdot (z + 0.044715 \cdot z^3))]$     |
|------------|---------------------------------------------------------------------------------------------------------|
| Türev      | $\Phi(z) + z \cdot \phi(z)$ [ $\Phi$ : kümülatif normal, $\phi$ : normal yoğunluk]                      |
| Aralık     | $(\approx -0.17, +\infty)$                                                                              |
| Avantaj    | BERT, GPT-2, GPT-3, ViT — modern transformer mimarilerinin standart tercihi. Stochastic regularizasyon. |
| Dezavantaj | Hesaplaması daha pahalı. ReLU/ELU'nun bazı basit görevlerde üstünlüğü olabilir.                         |

## 4. Vanishing Gradient Problemi — Matematiksel Kanıt

Backpropagation sırasında gradyanlar, çıkıştan girişe doğru katman katman zincir kuralıyla çarpılır. Her Sigmoid katmanında gradyan en fazla 0.25 ile çarpılır. n katmanlı derin bir ağda:

$$|\text{grad\_ilk\_katman}| \leq (0.25)^n \rightarrow n=10: \sim 9.5 \times 10^{-7}$$

| Katman Sayısı | Sigmoid Gradyanı      | Tanh Gradyanı | ReLU Gradyanı | Durum                |
|---------------|-----------------------|---------------|---------------|----------------------|
| 1             | 0.2500                | 0.7000        | 1.0000        | Normal               |
| 2             | 0.0625                | 0.4900        | 1.0000        | Normal               |
| 3             | 0.0156                | 0.3430        | 1.0000        | Normal               |
| 5             | 0.000977              | 0.1681        | 1.0000        | ⚠ Küçülüyor          |
| 10            | $9.5 \times 10^{-7}$  | 0.0282        | 1.0000        | ⚠ Sigmoid vanishing  |
| 15            | $9.3 \times 10^{-10}$ | 0.0047        | 1.0000        | ✗ Sigmoid ölü        |
| 20            | $\approx 0$           | 0.0008        | 1.0000        | ✗ Tanh de zorlanıyor |

ReLU'nun Çözümü:  $z > 0$  için türev her zaman 1.0  $\rightarrow$  gradyan katman boyunca taşınır. Bu nedenle ResNet, VGG, EfficientNet gibi çok derin ağlar ReLU ailesi kullanır.

## 5. Dead Neuron (Ölü Nöron) Problemi

---

ReLU aktivasyonunda  $z \leq 0$  olan bir nöron için çıkış sıfır, türev sıfırdır. Bu nöron backpropagation sırasında sıfır gradyan alır — ağırlıkları hiç güncellenmez ve nöron kalıcı olarak 'ölür'. Büyük öğrenme hızı veya hatalı başlatma bunu tetikler. Bazı ağlarda %50'ye kadar nöron ölebilir!

- Büyük öğrenme hızı  $\rightarrow$  ağırlıklar büyük güncelleme alır  $\rightarrow$   $z$  kalıcı negatif olur
- Çözüm 1: Leaky ReLU —  $z < 0$ 'da  $\alpha \cdot z$  çıktısı,  $\alpha$  türevi  $\rightarrow$  nöron canlı kalır
- Çözüm 2: ELU —  $z < 0$ 'da smooth üstel çıkış, türev  $\alpha$ 'ya yakın
- Çözüm 3: Öğrenme hızını düşürmek ( $lr < 0.01$ )
- Çözüm 4: He başlatma — ReLU için özel ağırlık başlatma:  $W \sim N(0, \sqrt{2/n})$

## 6. Framework Karşılaştırması

| Fonksiyon  | NumPy                                             | TensorFlow/Keras      | PyTorch            |
|------------|---------------------------------------------------|-----------------------|--------------------|
| Sigmoid    | $1/(1+\text{np.exp}(-z))$                         | activation='sigmoid'  | nn.Sigmoid()       |
| Tanh       | $\text{np.tanh}(z)$                               | activation='tanh'     | nn.Tanh()          |
| ReLU       | $\text{np.maximum}(0, z)$                         | activation='relu'     | nn.ReLU()          |
| Leaky ReLU | $\text{np.where}(z > 0, z, 0.01 * z)$             | layers.LeakyReLU(0.2) | nn.LeakyReLU(0.01) |
| ELU        | $\text{np.where}(z > 0, z, a * (\text{e}^z - 1))$ | layers.ELU(alpha=1.0) | nn.ELU(alpha=1.0)  |
| SELU       | $\lambda \cdot \text{ELU}(z, \alpha)$ [sabitler]  | activation='selu'     | nn.SELU()          |
| Swish/SiLU | $z * \text{sigmoid}(z)$                           | activation='swish'    | nn.SiLU()          |
| GELU       | $0.5z(1 + \tanh(\dots))$                          | activation='gelu'     | nn.GELU()          |

## 7. Aktivasyon Seçim Karar Rehberi

| Durum / Görev               | Önerilen Aktivasyon  | Neden?                                 |
|-----------------------------|----------------------|----------------------------------------|
| Çıkış — İkili sınıflandırma | Sigmoid              | (0,1) aralığı olasılık yorumu sağlar   |
| Çıkış — Çok sınıflandırma   | Softmax              | Tüm sınıfların toplamı=1 garantisi     |
| Çıkış — Regresyon           | Linear (yok)         | Sınırsız değer aralığı gerekli         |
| Gizli — Genel başlangıç     | ReLU                 | Hızlı, basit, çoğu durumda iyi         |
| Gizli — Derin CNN           | ReLU veya Leaky ReLU | Vanishing gradient yok, pratik         |
| Gizli — Transformer / NLP   | GELU                 | BERT, GPT-2/3/4 standart tercihi       |
| Gizli — EfficientNet tipi   | Swish (SiLU)         | ImageNet büyük ölçek testlerinde üstün |
| Çok derin FC ağlar          | SELU                 | Self-normalizing, Batch Norm gereksiz  |
| Dead neuron sorunu          | Leaky ReLU veya ELU  | $z < 0$ bölgede hâlâ gradyan akıyor    |
| RNN/LSTM gizli durum        | Tanh                 | Sıfır merkezli, $(-1,1)$ aralığı uygun |
| Bilmiyorum, ne yapayım?     | ReLU ile başla       | En güvenli ve yaygın başlangıç noktası |



## 8. Yaygın Hatalar ve Tuzaklar

|                                                             |                                                                                                                                  |
|-------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <b>Hata 1: Sigmoid'i gizli katmanda kullanmak</b>           | Derin ağılarda vanishing gradient sorununa yol açar. Sigmoid yalnızca çıkış katmanında ikili sınıflandırma için kullanılmalıdır. |
| <b>Hata 2: Aktivasyon seçmeden önce test etmemek</b>        | Her problem farklı aktivasyona yanıt verebilir. Başlangıçta ReLU kullan, ardından GELU/Swish ile karşılaştır.                    |
| <b>Hata 3: Tüm katmanlarda aynı aktivasyonu kullanmak</b>   | Çıkış katmanı göreve özel aktivasyon ister. Regresyon → Linear, İkili sınıf → Sigmoid, Çok sınıf → Softmax.                      |
| <b>Hata 4: Dead Neuron'u fark etmemek</b>                   | Eğitim kaybı azalmıyor, doğruluk düşük kalıyorsa nöronların öldüğünü düşün. Öğrenme hızını düşür veya Leaky ReLU'ya geç.         |
| <b>Hata 5: SELU'yu CNN'de kullanmaya çalışmak</b>           | SELU yalnızca fully-connected katmanlarda self-normalizing özelliğini gösterir. CNN'lerde Batch Norm + ReLU daha iyi çalışır.    |
| <b>Hata 6: Swish ve SiLU'nun aynı şey olduğunu bilmemek</b> | PyTorch'ta Swish, nn.SiLU() olarak adlandırılır. TensorFlow'da 'swish'. İkisi matematiksel olarak aynıdır.                       |

## 9. Özet Tablo

| Fonksiyon  | Formül                                | Aralık              | Maks. Türev | Modern Kullanım     |
|------------|---------------------------------------|---------------------|-------------|---------------------|
| Sigmoid    | $\sigma(z)=1/(1+e^{-z})$              | $(0,1)$             | 0.25        | Çıkış — ikili sınıf |
| Tanh       | $\tanh(z)$                            | $(-1,1)$            | 1.00        | RNN/LSTM gizli      |
| ReLU       | $\max(0,z)$                           | $[0,+\infty)$       | 1.00        | CNN gizli standart  |
| Leaky ReLU | $\max(\alpha z, z)$                   | $(-\infty,+\infty)$ | 1.00        | Dead neuron çözümü  |
| ELU        | $z$ veya $\alpha(e^z-1)$              | $(-\alpha,+\infty)$ | 1.00        | Derin gizli katman  |
| SELU       | $\lambda \cdot \text{ELU}(z, \alpha)$ | $(-1.76,+\infty)$   | 1.76        | Çok derin FC ağlar  |
| Swish      | $z \cdot \sigma(z)$                   | $(-0.28,+\infty)$   | $\sim 1.10$ | EfficientNet ailesi |
| GELU       | $0.5z(1+\tanh(\dots))$                | $(-0.17,+\infty)$   | $\sim 1.13$ | BERT, GPT, ViT      |

## 10. Kaynaklar ve İleri Okuma

- [1] Nair, V. & Hinton, G.E. (2010). Rectified Linear Units Improve Restricted Boltzmann Machines. ICML.
- [2] Maas, A.L. et al. (2013). Rectifier Nonlinearities Improve Neural Network Acoustic Models. ICML.
- [3] Clevert, D.A. et al. (2016). Fast and Accurate Deep Network Learning by Exponential Linear Units (ELUs). ICLR.
- [4] Klambauer, G. et al. (2017). Self-Normalizing Neural Networks. NeurIPS.
- [5] Ramachandran, P. et al. (2017). Searching for Activation Functions. arXiv:1710.05941. (Swish)
- [6] Hendrycks, D. & Gimpel, K. (2016). Gaussian Error Linear Units (GELUs). arXiv:1606.08415.
- [7] Goodfellow, I., Bengio, Y. & Courville, A. (2016). Deep Learning, MIT Press. — Bölüm 6.3.

**Sonraki Adım: Modül 03 — Kayıp Fonksiyonları ve Optimizasyon | MSE · BCE · Adam · SGD · Öğrenme Hızı Programları**